

Activity 4 : Design a High Level Architecture

4.1 System Architecture Diagram

The diagram below illustrates the high level architecture for the FitFlow redesign, built on the recommended technology stack: a React Native client layer, a Node.js/Express microservice backend behind an API gateway, a dedicated AI/ML microservice layer (on device TensorFlow Lite plus a cloud AI service and computer vision service), a data layer combining PostgreSQL and Firebase, a Redis caching layer, and Firebase Auth as a cross-cutting authentication service.

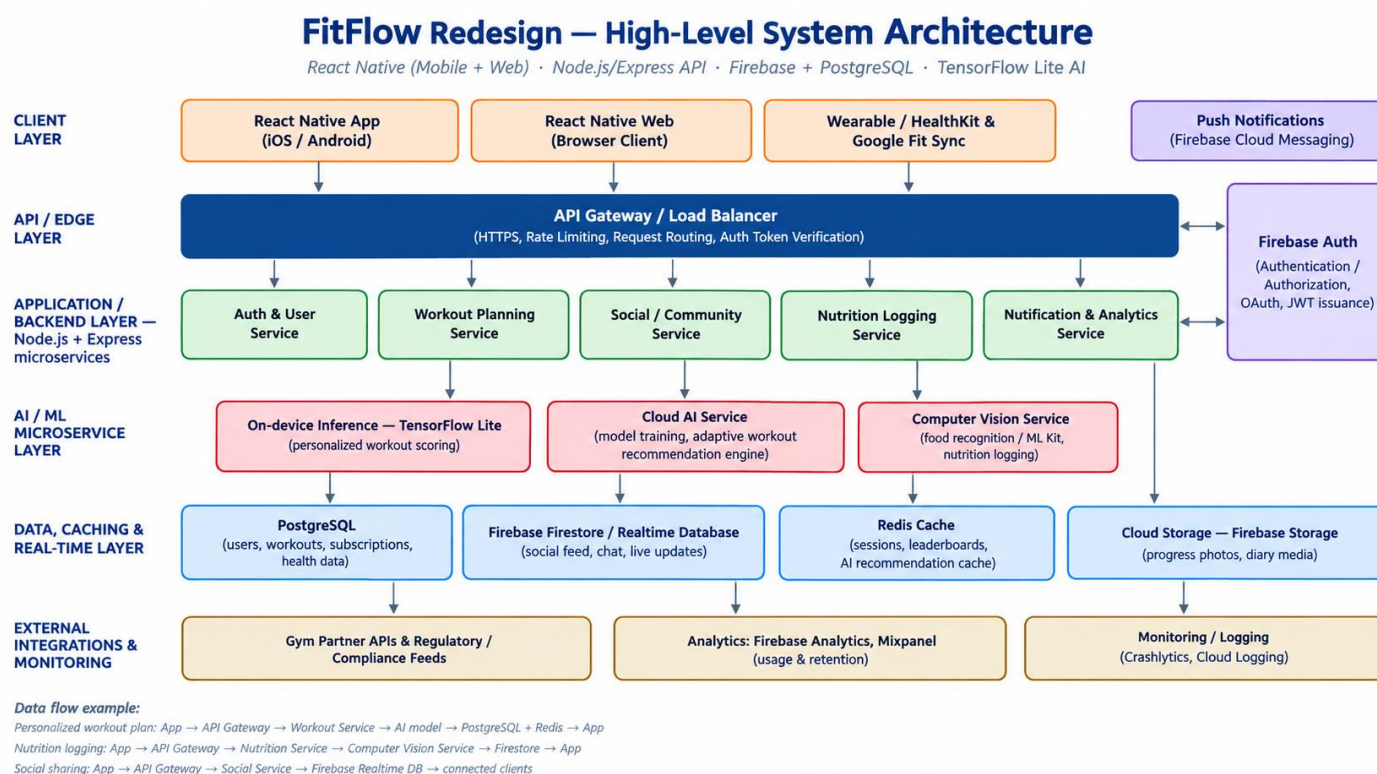


Figure 4.1: FitFlow high-level system architecture

4.2 Key Components

- **Client Layer:** React Native mobile app (iOS/Android), React Native Web client, and wearable/HealthKit/Google Fit sync.
- **API / Edge Layer:** API Gateway / load balancer handling HTTPS termination, rate limiting, routing, and auth token verification.
- **Application Layer:** Node.js/Express microservices Auth & User, Workout Planning, Social/Community, Nutrition Logging, and Notification & Analytics services.

- **AI / ML Microservice Layer:** On device TensorFlow Lite inference for low latency personalization, a Cloud AI service for model training and adaptive recommendations, and a Computer Vision service for camera based nutrition logging.
- **Data & Caching Layer:** PostgreSQL (structured/health data), Firebase Firestore/Realtime Database (social feed and live updates), Redis (caching), and Firebase Cloud Storage (media).
- **Cross cutting Services:** Firebase Auth issuing and verifying tokens across all services; Firebase Cloud Messaging for push notifications.
- **External Integrations:** Gym partner APIs, analytics (Firebase Analytics, Mixpanel), and monitoring/logging (Crashlytics, Cloud Logging).

4.3 Data Flows for Critical Features

- **Personalized Workout Plans:** The app requests a plan via the API Gateway, which routes to the Workout Planning Service; the service queries the on device TensorFlow Lite model for immediate suggestions and the Cloud AI service for deeper personalization, reading/writing plan data to PostgreSQL and caching results in Redis before returning the response to the app.
- **Social Sharing:** A post or update passes through the API Gateway to the Social Service, which writes to Firebase Firestore/Realtime Database; Firestore's listener model then fans the update out in real time to all connected clients subscribed to that feed or community group.
- **Nutrition Tracking:** A meal photo is sent through the API Gateway to the Nutrition Logging Service, which forwards the image to the Computer Vision Service for food recognition; the resulting log entry is written to Firestore and reflected back to the app almost instantly.

4.4 Security, Scalability, and Integration Considerations

Security

- All client-server traffic is encrypted via HTTPS/TLS; the API Gateway verifies JWT tokens issued by Firebase Auth before routing requests to internal services.
- Health sensitive data in PostgreSQL is protected with encryption at rest, row level security, and audit logging to support HIPAA/GDPR obligations; Firebase security rules restrict access to social/real time data by user and group.
- Rate limiting and request validation occur at the gateway to mitigate abuse; a signed Business Associate Agreement (BAA) with the cloud provider covers HIPAA regulated workloads, and only anonymized wellness data is exposed to secondary stakeholders such as healthcare-provider partners.

Scalability

- Node.js services are stateless and horizontally scalable behind the load balancer; Firebase scales automatically for real time and notification workloads.
- PostgreSQL uses read replicas for reporting/analytics queries, and Redis caches hot paths such as leaderboards and AI recommendation results to reduce database load.
- The AI/Cloud service scales independently of the core application services, decoupled through internal service calls so spikes in AI workload do not degrade core app responsiveness.

Integration

- Internal services communicate over REST (with gRPC as a future option for latency sensitive AI calls); the on device TensorFlow Lite model is periodically synced from the cloud training pipeline.
- External integrations (gym partner APIs, wearables, HealthKit/Google Fit) are isolated behind dedicated adapters at the service layer so that third party API changes do not ripple into core business logic.

4.5 Architecture Decision Record (ADR)

Field	Detail
ADR ID / Title	ADR-001: Core Technology Stack for the FitFlow Redesign
Status	Accepted
Context	FitFlow needed a redesign delivering AI personalized workouts, social community features, and improved nutrition tracking across iOS, Android, and Web, executed quickly and cost effectively by a mid sized team while meeting GDPR/HIPAA obligations for health data.
Decision	Adopt React Native (+RN Web) for the client, Node.js/Express microservices for the backend, PostgreSQL for structured/health data, Firebase (Firestore/RTDB, FCM, Auth) for real time social features, notifications and authentication, and a hybrid AI layer combining on device TensorFlow Lite with a Cloud AI service and computer vision service for nutrition logging.
Consequences (Positive)	Fastest development speed and lowest cost of the three evaluated options; single primary language (TypeScript) across client and backend; strong native fit for real time social features; clear path to AI/ML integration without requiring three separate native codebases.
Consequences (Trade-offs)	Slightly lower raw performance ceiling than fully native (Kotlin/Swift) apps, mitigated by React Native's New Architecture and selective native modules where profiling shows bottlenecks; reliance on two data stores (PostgreSQL and Firebase) requires a clear data ownership and consistency strategy.

Field	Detail
Alternatives Considered	Option B : Flutter + FastAPI + MongoDB + Auth0 rejected due to team fragmentation across two languages and no native real time data layer. Option C: Kotlin Multiplatform + Go + DynamoDB + Cognito rejected due to the slowest development speed, weakest AI/ML ecosystem fit, and highest long term maintenance burden despite superior raw performance.

Table 4.1: Architecture Decision Record for the FitFlow technology stack